Removing duplicates:

```
#removing duplicates

list = [1, 1, 2, 2, 3, 3, 3, 8, 7, 8, 0, 2, 0]
final_list =[]

for i in list:
  if i not in final_list:
    final_list.append(i)

print(final_list)
```

```
[1, 2, 3, 8, 7, 0]
```

Time complexity = O(n^2)
Space complexity = O(n)

Day 2

String immutability:

Strings in Python are immutable, i.e, they can't be changed once created. It keeps them safe, consistent and efficient.
Immutability makes strings hashable, memory-efficient, thread-safe, ensuring they can be reused without unexpected changes.

Benefits of immutable objects:

1. **Hashability**: immutable objects can be dictionary keys since their hash never changes.
2. **Memory-efficiency**: reused by Python, saving memory.
3. **Thread-safety**: safe in multithreading as they can't be modified.
4. **Predictability**: values stay fixed, making code easier to debug.
5. **Performance**: support faster lookups with cached hash values.

String traversal:

It refers to the process of one character at a time, often started at the beginning, select each character in turn, do something to it and continue until the end.
Can be done using the '***for()***' loop -

```
text = "Hello, World!"

for char in text:
    print(char)
```

Output

```
H
e
l
l
o
,
W
o
r
l
d
!
```

String slicing:

It is a way to get specific parts of a string by using start, end and step values.

- It is used for text manipulation.

Syntax: substring = s[start: end: step]

s -> original string

Start -> starting index (inclusive)

End -> ending index (exclusive)

Step -> interval between indexes

Printing every second/third character of the string:

```
s = "abcdefghi"

# Every second character
print(s[::2])

# Every third character from index 1 to 8 (exclusive)
print(s[1:8:3])
```

Output

```
acegi
beh
```

Lists v/s Strings:

List	String
[a, b, c]	“abc” or ‘abc’
Mutable	Immutable
Can hold multiple data types	Only characters
Iterates over elements	Iterates over characters
Not hashable	Hashable
Use: collection of data that change	Use: text data

String Concatenation:

It allows us to combine two or more strings into one.

- Can be done using the “**+**” operator.

```
s1 = "Hello"
s2 = "World"
res = s1 + " " + s2
print(res)
```

Output

```
Hello World
```

- Can also be done using the '***join()***' function, when working with sequence of strings (list/tuple)

```
a = ["Python", "is", "a", "popular", "language", "for", "programming"]

# Use join() method to concatenate list elements
# into a single string, separated by spaces
res = " ".join(a)
print(res)
```

Output

```
Python is a popular language for programming
```

String spacing:

Can be done using the '***format()***' method

- For filling space in the **right**: **'{: <no.of spaces}'.format(stringName)**
- For filling space in the **left**: **'{: >no.of spaces}'.format(stringName)**
- For filling space in **both left and right**: **'{: ^no.of spaces}'.format(stringName)**

```
# original string
string = "GFG"

# Right Padding of the string
right_padding = ('{: <5}'.format(string))
print(f'\"{right_padding}\"')

# Left Padding of the string
left_padding = ('{: >5}'.format(string))
print(f'\"{left_padding}\"')

# Center Padding of the string
central_padding = ('{: ^5}'.format(string))
print(f'\"{central_padding}\"')
```

Output:

```
"GFG  "
"  GFG"
" GFG "
```

Reversing a string:

```
#reverse a string

str = "Anirudh Choudhary"
print("string before reversing: ", str)

new_str = str[::-1]
print("string after reversing: ", new_str)
```

```
string before reversing:  Anirudh Choudhary
string after reversing:  yrahduohC hdurinA
```

To check if string is a palindrome or not:

```
#to check if string is a palindrome

str = input("Enter a string: ")
rev = ''

for char in str:
  rev = char + rev
if str == rev:
  print("String IS a palindrome.")
else:
  print("String IS NOT a palindrome.")
```

```
Enter a string: level
String IS a palindrome.
```

```
#to check if string is a palindrome

str = input("Enter a string: ")
rev = ''

for char in str:
  rev = char + rev
if str == rev:
  print("String IS a palindrome.")
else:
  print("String IS NOT a palindrome.")
```

```
Enter a string: mango
String IS NOT a palindrome.
```

Counter character frequencies in a string:

1. Upper-case and lower case characters are counted different.

```
#counting character frequencies

a = input("enter a string: ")
dict = {}

for char in a:
  if char not in dict:
    dict[char] = 1
  else:
    dict[char] += 1

print(dict)
```

```
enter a string: Australia
{'A': 1, 'u': 1, 's': 1, 't': 1, 'r': 1, 'a': 2, 'l': 1, 'i': 1}
```

2. Upper-case and lower case are counted same.

```
#counting character frequencies

a = input("enter a string: ")
dict = {}

for char in a.lower():
  if char not in dict:
    dict[char] = 1
  else:
    dict[char] += 1

print(dict)
```

```
enter a string: Australia
{'a': 3, 'u': 1, 's': 1, 't': 1, 'r': 1, 'l': 1, 'i': 1}
```

Finding second largest value in an array:

```
#finding second largest number in an array

arr = list(map(int, input("enter numbers: ").split(",")))
if arr[0] > arr[1]:
  max1 = arr[0]
  max2 = arr[1]
else:
  max1 = arr[1]
  max2 = arr[0]
for i in arr[2:]:
  if i > max1:
    max2 = max1
    max1 = i
  elif i > max2:
    max2 = i

print("largest element: ", max1)
print("second largest element: ", max2)
```

```
enter numbers: 1, 3, 10, 15, 13, 21
largest element:  21
second largest element:  15
```
